

Latent-Space Action Optimization under Implicit Constraints for Robotic Manipulation

Yi Wang¹, Ko Ayusawa², Gentiane Venture¹, and Eiichi Yoshida³

Abstract—Robot manipulation tasks can often be formulated as a constrained optimization problem, where the feasibility constraints are imposed by dynamics, object contacts, and kinematic requirements. However, effectively formulating these implicit and hybrid constraints for the real-world tasks presents significant challenges. In this work, we propose a data-driven reparameterization approach that enables optimization motion planning without explicit constraint specification. Given a feasible demonstration set, we train a Transformer-based Conditional Variational Autoencoder (CVAE) model, where the decoder acts as a differentiable generator that maps latent vectors to feasible action consistent with the current visual observation. Planning is formulated as an optimization over the latent variable with respect to an additional task objective specified by the user, and can be solved using standard gradient-based optimizers. To maintain consistency with the learned statistical representation during optimization and thereby effectively restrict the search to the data-driven action constraints, we introduce a prior-aware penalty term in optimization inference. Experiments on the real-world and contact-rich block assembly task demonstrate that the learned decoder preserves constraints’ feasibility at high rates of about $\geq 97\%$ across all phases. In the online optimization inference, latent space optimization improves motion quality by extra objective function setting, such as reducing insertion-phase jerk by 14.61% and acceleration by 10.84% compared to unoptimized latent sampling.

I. INTRODUCTION

Unlike free-space motion planning, robotic manipulation involves intricate interactions with objects in the environment, which can often be expressed as constraints optimization problem [1]. Consider a block-building task, as illustrated in Fig. 1. The robot must transport the block while maintaining safe clearance from the table and surrounding blocks, precisely align it with the target placement, and then place and release it at a specific angle and force, with minimal disturbance to the partially built structure.

However, reliably specifying such constraints across a wide range of real-world tasks remains highly challenging. A common and intuitive approach is to explicitly encode kinematic, dynamic, and contact-related constraints using the robot-object relative pose [2], [3] and object 3D point model; yet, accurately modeling physical interactions is

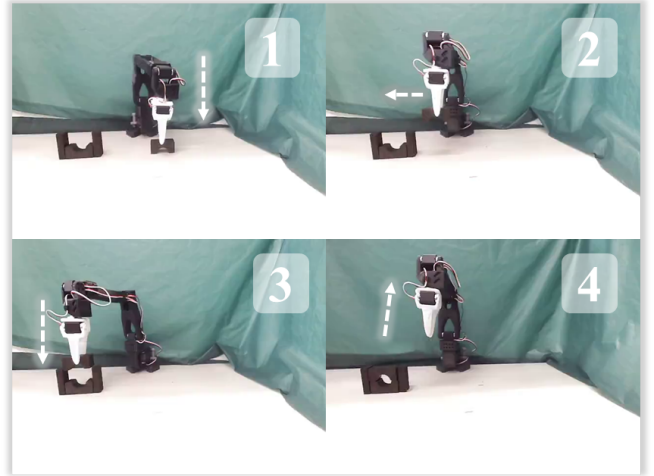


Fig. 1: Block-Building Manipulation: (1) the gripper approaches and grasps a wooden block; (2) the robot transports the block toward the partially built structure; (3) the robot places the block with a controlled downward motion. (4) the robot opens the gripper and returns without touching the blocks. Dashed arrows indicate the intended motion direction.

labor-intensive and often brittle under uncertainty and environmental variation. Moreover, beyond physical feasibility, manipulation in human environments must also be adapted to context-dependent human preferences. Data-driven methods alleviate manual modeling by learning from demonstrations. For example, end-to-end visuomotor policies map observations directly to actions [4], [5] and implicitly capture rich interaction patterns with fast and lightweight inference. However, it is lack of interpretability and nontrivial to incorporate new requirements or preference terms without substantial fine-tuning for them. Recent hybrid approaches also combine representation learning with optimization planning by obviously form action features [6]–[8]. While this improves flexibility and coverage across tasks, incorporating learned and perception-conditioned explicit math constraints into trajectory optimization often leads to high-dimensional and highly nonconvex problems, making optimization computationally expensive and sensitive to initialization. Therefore, we ask whether there exists a constraint representation that can (i) capture such complex constraints and adapt to visual feedback (ii) remain compatible with the constrained-optimization framework to support the smooth integration of diverse cost functions, and (iii) be solved efficiently using

¹Yi Wang and Gentiane Venture are with the Graduate School of Engineering, The University of Tokyo, Japan. {yiw1022, venture}@g.ecc.u-tokyo.ac.jp

²Ko Ayusawa is with the Research Institute on Human and Societal Augmentation, National Institute of Advanced Industrial Science and Technology (AIST), Kashiwa, Chiba, Japan. k.ayusawa@aist.go.jp

³Eiichi Yoshida is with the Department of Medical and Robotic Engineering Design, Tokyo University of Science, Japan. eiichi.yoshida@rs.tus.ac.jp

standard gradient-based optimization and modern numerical optimizers.

In our paper, we adopt a reparameterization based formulation for constrained manipulation planning. We train a conditional VAE on task-specific feasible trajectory demonstrations from a human expert, introducing latent variables that replace the original action variables. The decoder serves as a differentiable reparameterization map from latent parameters to a feasible action sequence, enabling planning and control by optimizing in the learned latent space. We condition on visual observations and the current robot state to capture scene-dependent feasible sets. During deployment, we use receding horizon control to handle disturbances and model mismatch. We further regularize latent updates to remain in the high-density region of the learned latent distribution, keeping decoded trajectories within the feasible set observed in training. In the experiments, we demonstrated the effectiveness of our approach by a block-assembling task example in the real world, which needs to meet relatively strict pose and mechanical constraints.

Our contributions are summarized as follows:

- 1) **Reparameterization-based constrained planning via learning.** We propose a data-driven reparameterization framework that replaces optimization in the raw action space with optimization in a learned latent space. A CVAE trained on feasible demonstrations serves as an implicit constraint model, where the decoder acts as a differentiable mapping that parameterizes the constraint-satisfying space.
- 2) **Receding-horizon control with vision for robust execution.** We condition feasibility on visual observations and combine latent-space optimization with receding-horizon control, using once-per-window visual feature extraction and periodic replanning for robustness to environment disturbing and variety.
- 3) **Latent distribution regularization for feasibility preservation.** We design a latent regularization term that penalizes deviation from typical regions of the learned latent distribution, encouraging optimization to remain within the demonstrated feasible space and improving reliability during test-time search.

II. BACKGROUND AND RELATED WORKS

A. Learning from Demonstrations for Manipulation

Robotic manipulation tasks can be viewed as the problem of representing and generating behaviors that satisfy task objectives, physical feasibility, and interaction constraints imposed by the environment. Complex and changeable environments and tasks make action feature representation a central challenge.

Learning from demonstrations (LfD) provides an effective data-driven solution [9] by extracting such a representation directly from expert examples, without requiring full manual specification. Depending on how the demonstrated knowledge is represented, existing LfD methods for manipulation can be broadly categorized into two paradigms: end-to-end

policies that directly map observations to actions [4], [5], [10]–[13], and hybrid methods that learn direct or latent representations from demonstrations and perform optimization in the learned space before mapping the solution back to the original action space [6]–[8], [14]–[19].

Therefore, a key gap remains between these two paradigms: end-to-end policies are effective at learning complex manipulation patterns from demonstrations, but provide limited flexibility for incorporating new optimization objectives at test time; hybrid optimization-based methods offer better controllability and task adaptability, but often lead to high-dimensional and highly nonconvex optimization problems. Our target is to simultaneously provide (i) a latent representation of feasibility under contact-rich constraints and (ii) a efficient, practical and flexible interface for downstream optimization planning.

B. Constrained Optimization for Robot Control

Let $x_t \in \mathbb{R}^{n_x}$ denote the robot state at time step t and $u_t \in \mathbb{R}^{n_u}$ denote the control input. Given a planning horizon T , a robot dynamic model is:

$$x_{t+1} = f(x_t, u_t, t), \quad x_0 \in \mathbb{R}^n \quad (1)$$

We have the predefined initial configuration and the constraints settings, aiming to plan an optimal trajectory and controls to reach a target:

$$\begin{aligned} \min \quad & \sum_{t=0}^{T-1} \ell(x_t, u_t) + \ell_T(x_T) \\ \text{s.t.} \quad & x_{t+1} = f(x_t, u_t), \quad t = 0, \dots, T-1, \\ & g_i(x_{0:T}, u_{0:T-1}) \leq 0, \quad i = 1, 2, \dots, l, \\ & h_j(x_{0:T}, u_{0:T-1}) = 0, \quad j = 1, 2, \dots, p. \end{aligned} \quad (2)$$

where $\ell(\cdot)$ and $\ell_T(\cdot)$ are stage and terminal costs, and $g_i(\cdot)$ and $h_j(\cdot)$ are inequality and equality constraint functions, respectively. During a manipulation task, constraints arise from combinatorial requirements of the robot's actions in relation to the environment, which are often difficult to model explicitly and accurately.

C. β -VAE for Trajectory Modeling

We model robot behavior by learning a generative distribution over feasible trajectories using a β -VAE. Specifically, we introduce a low-dimensional latent variable z and define a Gaussian latent-variable model with a standard normal prior

$$p(z) = \mathcal{N}(0, I), \quad (3)$$

VAE is composed of an encoder and a decoder:

$$q_\phi(z | x) = \mathcal{N}(z; \mu_\phi(x), \text{diag}(\sigma_\phi^2(x))), \quad (4)$$

where $\mu_\phi(x)$ and $\sigma_\phi(x)$ are produced by Encoder with parameters ϕ . The decoder models the conditional likelihood with a Gaussian distribution:

$$p_\theta(x | z) = \mathcal{N}(x; \mu_\theta(z), \sigma_x^2 I) \quad (5)$$

Learning proceeds by maximizing the evidence lower bound (ELBO), which decomposes into a reconstruction term and a KL regularization term. For β -VAE, we minimize

$$\mathcal{L}_\beta(x) = \mathbb{E}_{q_\phi(z|x)} [-\log p_\theta(x|z)] + \beta \text{KL}(q_\phi(z|x) \parallel p(z)), \quad (6)$$

where the first term is a reconstruction loss and the second term regularizes the posterior to remain close to the prior. The coefficient β controls the trade-off between reconstruction accuracy and latent regularization. In the context of trajectory modeling, it is important to choose a suitable β to formulate the latent space to support stable gradient-based search over z , while still capturing the dominant modes of feasible behavior.

D. Problem Statement

We consider manipulation planning over a finite horizon of length T . At time t , the robot receives an observation o_t and plans control action sequence $\{a_{t+\ell}\}_{\ell=1}^T$ with $a_t \in \mathbb{R}^d$. We denote the corresponding action horizon by

$$A_t \triangleq a_{t+1:t+T} \in \mathbb{R}^{T \times d}. \quad (7)$$

In manipulation, constraints are various on both the scene and the interaction phase, so we formalize them using a time and context dependent feasible set

$$\mathcal{F}_t(o_t) \triangleq \{A \in \mathbb{R}^{T \times d} \mid g_{j,t}(A; o_t) \leq 0, j = 1, \dots, m_t\}, \quad (8)$$

where $g_{j,t}(\cdot)$ may encode kinematic limits, collision avoidance, and contact consistency, and can vary across time and context.

To capture the constraints structure of $g_{j,t}(\cdot)$, we first collect a set of feasible demonstrations $\mathcal{D}_{\text{demo}} = \{\tau^{(i)}\}_{i=1}^N$, where each demonstration trajectory $\tau^{(i)} = \{(o_t^{(i)}, a_t^{(i)})\}_{t=1}^{L_i}$ is recorded at a fixed control rate. Given a dataset of expert feasible demonstrations, our goal is to learn a differentiable reparameterization map

$$f_\theta : \mathbb{R}^k \times \mathcal{O} \rightarrow \mathbb{R}^{T \times d}, \quad \hat{A}_t = f_\theta(z; o_t), \quad (9)$$

such that decoded output sequences remain feasible under the corresponding constraints

$$f_\theta(z; o_t) = \mathcal{F}_t(\hat{A}_t) \in \mathcal{F}_t(o_t). \quad (10)$$

It enables planning by optimizing in the latent space with a task objective $J(\cdot)$

$$z^* = \arg \min_{z \in \mathbb{R}^k} J(f_\theta(z; o_t)), \quad A_t^* = f_\theta(z^*; o_t), \quad (11)$$

where $J(\cdot)$ can include multiple objective terms about user's preference.

III. METHOD

Our method is designed to recover a latent representation of unknown constraints for the manipulation planning task. With the latent representation, we can plan the action based on the extra objective function without constraints formulation. As shown in Fig2, it trains a Transformer-based CVAE

to model the conditional distribution of feasible action trajectories with the current visual observation and robot state. With the learned decoder as a reparameterization mapping, planning is performed by direct optimal control in latent space. This section details the technical implementation of our method.

A. Feasible Action Dataset Construction

For each demonstration trajectory $\tau^{(i)} = \{(o_t^{(i)}, a_t^{(i)})\}_{t=1}^{L_i}$ defined in Section II, we noted the observation that integrates synchronized multi-view visual feedback and the current robot proprioceptive configuration

$$o_t \triangleq \left(\{I_t^{(m)}\}_{m=1}^M, x_t \right), \quad (12)$$

where $\{I_t^{(m)}\}_{m=1}^M$ are RGB images captured from different M viewpoints and x_t is the robot state.

For the manipulation task, we represent the robot state configuration as

$$x_t = [\mathbf{q}_t, g_t] \in \mathbb{R}^d, \quad (13)$$

where $\mathbf{q}_t$ denotes the joint angles and g_t denotes the gripper state. Many manipulation systems expose low-level controllers that track desired joint targets together with a gripper command. To align the learning target with this interface, we focus on robot state x and let $a_t \triangleq x_t$.

To learn the mapping in Eq. 10, each action sequence $A_t^{(i)}$ in $\mathcal{D}$ must be feasible for constraints with respect to its corresponding start observation o_t . Therefore, we construct training pairs (o_t, A_t) , where o_t provides the initial scene and robot state and A_t is a feasible future action sequence from that state. These pairs are used to fit a conditional model over feasible action horizons, which is subsequently used for planning and control.

B. Reparameterization via Transformer-CVAE

In constrained manipulation, a single observation can correspond to multiple feasible action horizons that satisfy the same kinematic and dynamic constraints. A deterministic policy may average over these modes, reducing action diversity and making subsequent optimization less effective. We therefore model feasible action generation with a conditional variational autoencoder (CVAE), where the latent variable $z \in \mathbb{R}^k$ represents high-level intent and demonstration-specific variation within the feasible action set.

During training, the encoder $q_\phi(z \mid A_t, o_t)$ approximates the posterior over z given a demonstrated action A_t under the observation context o_t . The image is processed by a pretrained ResNet-18 backbone to obtain visual tokens, while current robot state is projected by a linear layer into a proprioceptive token. The decoder $p_\theta(A_t \mid z, o_t)$ can parameterize a differentiable trajectory generator that maps (z, o_t) to a distribution over action sequences. Intuitively, this formulation allows the model to represent multiple feasible behaviors for the same observation. For a fixed o_t , different latent samples z can be decoded to yield diverse action horizons that remain consistent with the constraints reflected in the demonstrations.

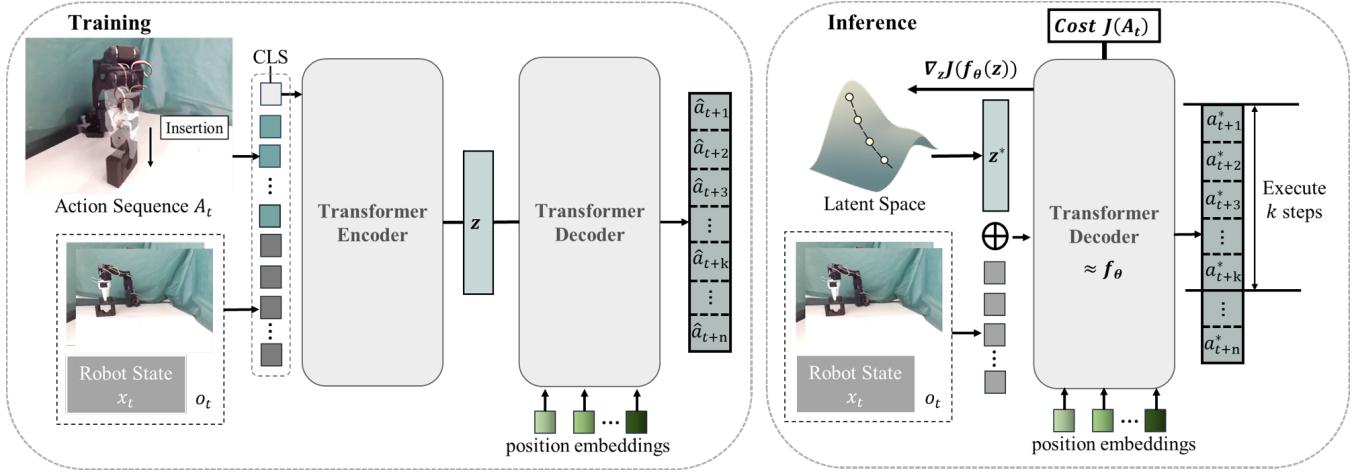


Fig. 2: Overview of Our Method. Left: The CVAE contains an Encoder and Decoder. It is trained with observation as conditions and action sequences. Right: In the inference, the decoder is the differentiable mapping in the latent space optimization. We optimize z^* with a plug-and-play objective and gradient from the decoder. z^* will be combined with observation tokens and pass the decoder to get the planning action sequence. In the real world, we only execute k steps.

Therefore, we can use the decoder mean as a deterministic reparameterization map

$$\hat{A}_t = f_\theta(z; o_t), \quad (14)$$

so that updating z directly changes the generated action A_t through f_θ .

C. Latent-space Gradient Planning

After training, the Transformer-CVAE decoder defines a differentiable reparameterization of a feasible action sequence under the current observation. The current observation o_t , consisting of camera images and proprioceptive states, is encoded into visual tokens by a pretrained ResNet-18 backbone and proprioceptive tokens by a linear projection. Conditioned on these observation tokens, the frozen decoder f_θ maps a latent code $z \in \mathbb{R}^k$ to a T -step action sequence:

$$\hat{A}_t = f_\theta(z; o_t) \in \mathbb{R}^{T \times d}, \quad (15)$$

where θ is fixed at inference. As shown in Algorithm 1, we perform planning directly in the latent space by treating z as the decision variable. This leverages the low dimensionality of z while keeping the generated actions on the learned feasible space.

a) *Extra Task objective*: Given a differentiable task objective $J_{\text{task}}(\cdot)$ defined on the decoded horizon, we solve

$$z^* = \arg \min_{z \in \mathbb{R}^k} J_{\text{task}}(f_\theta(z; o_t), o_t) + \lambda_z J_{\text{prior}}(z), \quad (16)$$

and set $A_t^* = f_\theta(z^*; o_t)$. Since f_θ is differentiable, gradients of the objective backpropagate through the decoder, enabling gradient-based latent updates. In practice, we can apply an existing optimizer such as Adam. Importantly, only z is optimized during planning, while the network parameters remain frozen.

b) *Latent prior regularization*: Optimizing z solely for task performance can push it into regions rarely visited during training, where the decoder may extrapolate and produce trajectories that violate implicit feasibility patterns. To keep optimization within the region of validity of the learned model, we regularize z to remain close to the typical latent distribution induced by demonstrations.

Concretely, we fit a diagonal Gaussian to the latent posterior statistics produced by the encoder over the training dataset, yielding mean $\bar{\mu} \in \mathbb{R}^k$ and standard deviation $\bar{\sigma} \in \mathbb{R}^k$. We then define a diagonal Mahalanobis penalty [20]

$$J_{\text{prior}}(z) \triangleq D_M(z; \bar{\mu}, \bar{\sigma}) = \frac{1}{k} \sum_{i=1}^k \frac{(z_i - \bar{\mu}_i)^2}{\bar{\sigma}_i^2}. \quad (17)$$

This term penalizes deviations along dimensions where the training distribution has low variance more strongly, discouraging the planner from exploiting brittle latent directions. When $\bar{\mu} = 0$ and $\bar{\sigma} = \mathbf{1}$, this reduces to isotropic L_2 regularization.

c) *Receding-horizon planning with warm-start*: To handle external environmental disturbances, we execute the planner in a receding-horizon manner. At each planning window, we optimize z using the latest observation o_t , decode a horizon A_t^* , execute only the first K actions with $K \leq T$, and then replan from the updated observation. This design retains multi-step lookahead while remaining reactive through frequent replanning. We additionally warm-start each optimization by initializing z with the optimized latent from the previous planning window since consecutive horizons overlap in time.

IV. EXPERIMENTS

A. Research Questions and Evaluation Rationale

As described in Section III, instead of hand-coding feasibility constraints for contact, collisions, and phase-dependent

Algorithm 1: Latent-space receding-horizon planning with warm-started gradient optimization

Input: decoder f_θ (fixed at inference); task cost C_{task} ; planning horizon T ; execute length $K \leq T$; current observation o_t ; gradient steps N ; step size α ; prior weight λ_z ; latent stats $(\bar{\mu}, \bar{\sigma})$; max replans H ; optional threshold ϵ .

Output: executed actions $\mathcal{A}$

```

 $\mathcal{A} \leftarrow [];$ 
 $z_{\text{prev}} \leftarrow \emptyset;$ 
for  $h \leftarrow 1$  to  $H$  do
    // Initialize latent
    if  $z_{\text{prev}} \neq \emptyset$  then
         $z \leftarrow \text{STOPGRAD}(z_{\text{prev}});$ 
    else
         $z \leftarrow \mathbf{0}_k;$  // or sample  $z \sim \mathcal{N}(0, I)$ 
    // Gradient-based latent optimization
    for  $i \leftarrow 1$  to  $N$  do
         $\hat{A}_t \leftarrow f_\theta(z; o_t);$ 
         $J \leftarrow C_{\text{task}}(\hat{A}_t, o_t) + \lambda_z D_M(z; \bar{\mu}, \bar{\sigma});$ 
         $z \leftarrow \text{OPTIMIZERSTEP}(z, \nabla_z J; \alpha);$ 
        if  $J \leq \epsilon$  then
            break
    // Execute first  $K$  actions and re-observe
     $A_t^* \leftarrow f_\theta(z; o_t);$ 
    for  $j \leftarrow 1$  to  $K$  do
         $\text{EXECUTE}(A_t^*[j]);$ 
         $o_t \leftarrow \text{OBSERVE}();$ 
         $\mathcal{A} \leftarrow \mathcal{A} \cup [A_t^*[j]];$ 
     $z_{\text{prev}} \leftarrow z;$ 
return  $\mathcal{A};$ 

```

interaction, we learn a feasibility reparameterization from expert demonstrations and perform gradient-based planning in a low-dimensional latent space conditioned on visual observations. To verify our method’s efficiency and performance, we structure the experiments around the following research questions.

RQ1: Feasibility learned by the reparameterization.

When sampling z from the prior, do decoded actions $\hat{A}_t = f_\theta(z; o_t)$ remain feasible under phase-dependent constraints?

RQ2: Objective-driven performance. Does our latent-space optimization planning reduce task cost and work for user-defined objectives?

RQ3: Online efficiency of latent optimization. For comparable task success and objective performance, does optimizing in the latent space yield accepted planning latency?

B. Experimental Setup

a) Robot Setup: We conduct our experiments using a single 3D-printed **LeRobot SO101** 6-DOF robotic arm [21] equipped with a two-finger gripper. As shown in Fig 3, the



Fig. 3: Robot & Hardware Setup

vision system includes two Angetube 920 1080P Stream Webcams: one positioned directly in front of the robotic arm, and the other positioned to the side and front, to supplement depth visibility and reduce occlusion. The capture resolution was set to 640×480 pixels.

b) Task Setup: As shown in Fig. 4, we evaluate our method on a contact-rich block stacking example task with rich constraints. The scene contains an arch-shaped block A to be manipulated, an inverted arch-shaped block B already placed, and two upright square blocks on the both sides that must not be disturbed. The goal is to place A such that the semicircular arcs of A and B align to form a circle, while keeping the side blocks stable.

The task is decomposed into four phases, each associated with distinct constraints.

- **Approach and Grasp.** The gripper moves above and grasps the block A . Constraints include maintaining a vertically downward end-effector orientation, avoiding collisions with the table and surrounding blocks, and keeping sufficient clearance while approaching.
- **Transport and insertion pre-alignment.** The robot transports A toward the partially built structure and aligns A relative to block B . Constraints include maintaining a vertically downward gripper orientation, keeping A parallel to B , and avoiding contact with the side blocks.
- **Insertion.** The robot inserts A and releases. Constraints include predominantly vertical motion of the end-effector and limited insertion speed to reduce impact and disturbance. In our setup, object disturbance should remain within approximately 2mm during alignment and insertion to avoid destabilizing the side blocks.
- **Open the gripper and leave.** The robot opens the gripper and returns to one pose. Constraint includes the gripper pose not contacting the placed block or the surrounding structure while leaving.

c) Model and Training: In our article, the specific structure of Transformer-CVAE was adapted from ACT [10]. All processing is trained and inferenced for 5,000 gradient steps on a single NVIDIA RTX 3090 GPU using the AdamW optimizer with a learning rate of 1×10^{-5} and a batch size of 16.

d) Data Collection: Each episode took approximately 10–15 seconds for a human operator to complete, which corresponds to about 200–400 time steps at a 30 Hz control rate. We collected 84 feasible demonstrations. Consequently, the total demonstration duration for the example task is

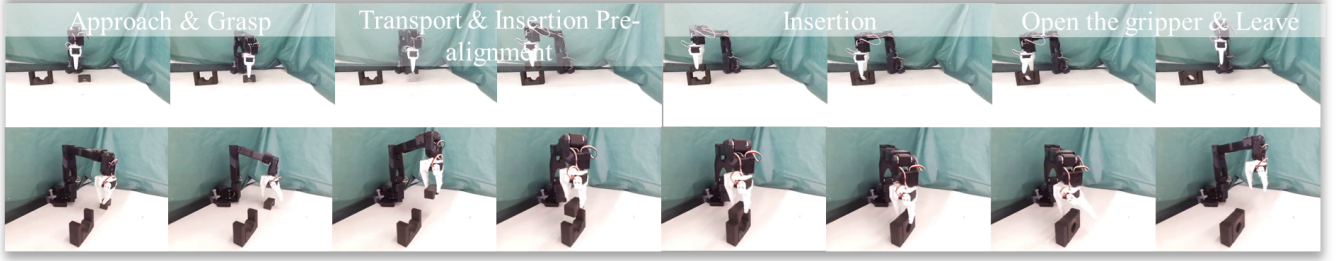


Fig. 4: Task Setup Example

TABLE I: Decoder feasibility results

Metric	Approach	Grasp	Transport	Insertion
Feasibility rate (%)	99.3	99.3	97.6	97.0

roughly 22 minutes; however, accounting for environment resets and occasional teleoperation failures, the actual wall-clock time was approximately 120 minutes.

C. Experiment Results

1) *Decoder feasibility under fixed context and latent variation:* We randomly select 40 different latent variables for 25 different object position. For each reset, we keep the fixed initial observation and independently sample latent vectors $z \sim \mathcal{N}(0, I)$. The decoded action is marked feasible if it satisfies all constraints and achieves the task in different phases.

As shown in Table I, the decoder produces actions that satisfy the phase-aligned constraints with high probability across all phases, indicating that feasibility is largely preserved under latent variation. The *Approach* and *Grasp* phases achieve constraints feasibility 99.3%. In contrast, feasibility drops slightly in the later phases, with *Transport* at 97.6% and *Insertion* at 97.0%. Its trend can be caused by that the later phases are more sensitive. During *Transport*, the carried block must remain parallel to B , so even mild lateral drift can cause a collision and affect the next step. During *Insertion*, the motion is further constrained to be vertical and executed at low speed, hard to correct misalignment once the end-effector enters the narrow insertion region. Nevertheless, feasibility remains high overall ($\geq 97\%$), suggesting that the decoder largely preserves phase-consistent constraint satisfaction despite latent variation.

Furthermore, it is observed if the initial localization fails and the robot cannot successfully grasp the object, the robot will re-localize based on visual feedback, rather than following a fixed behavioral pattern, which demonstrates the influence of observation tokens on action planning and shows that our method can adapt to different environmental settings to some extent.

2) *Latent-Space Optimization Effects:* We define cost terms that minimize the smoothness-oriented objective. The composite cost function is defined as follows:

$$z^* = \arg \min_z J(z) = w_{\text{vel}} J_{\text{vel}} + w_{\text{acc}} J_{\text{acc}} + w_{\text{jerk}} J_{\text{jerk}}, \quad (18)$$

TABLE II: The mean of absolute joint jerk ($\overline{|\ddot{q}|}$), acceleration ($\overline{|\dot{q}|}$), and velocity ($\overline{|\dot{q}|}$) for each joint DoF in the Insertion phase. Lower is better ($\downarrow$).

Var	DoF	$\overline{ \ddot{q} } (\downarrow)$	$\overline{ \dot{q} } (\downarrow)$	$\overline{ \dot{q} } (\downarrow)$
z_{rand}	0	767.54 ± 460.75	14.65 ± 8.83	0.40 ± 0.27
	1	383.82 ± 317.80	6.83 ± 5.95	0.15 ± 0.17
	2	8281.70 ± 1345.67	149.19 ± 26.16	6.94 ± 1.68
	3	4600.03 ± 669.95	85.44 ± 10.97	6.05 ± 1.52
	4	346.14 ± 142.97	6.84 ± 2.63	0.21 ± 0.09
	5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
	Avg	2396.54 ± 489.52	43.82 ± 9.09	2.29 ± 0.62
z^*	0	524.64 ± 477.61	9.56 ± 8.76	0.29 ± 0.22
	1	150.16 ± 282.83	3.34 ± 6.29	0.06 ± 0.10
	2	7113.10 ± 1363.79	135.50 ± 27.10	8.62 ± 1.18
	3	4454.42 ± 952.95	85.18 ± 16.50	8.06 ± 1.26
	4	36.63 ± 103.60	0.81 ± 2.30	0.02 ± 0.06
	5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
	Avg	2046.49 ± 530.13	39.07 ± 10.16	2.84 ± 0.47

where J_{vel} , J_{acc} and J_{jerk} are the means of the squared first-, second-, and third-order finite differences of the predicted joint angles, averaged over joints and time steps. We fix $w_{\text{vel}} : w_{\text{acc}} : w_{\text{jerk}} = 2 : 3 : 5$.

Since the *Insertion* phase is the most sensitive stage of the task, where abrupt changes in acceleration and jerk are most likely to induce impact forces and vibrations to disturb the partially built structure. To directly quantify this risk, we report the mean of the absolute joint accelerations and jerks over the insertion time window from the real observation with 30Hz, in 30 different environments.

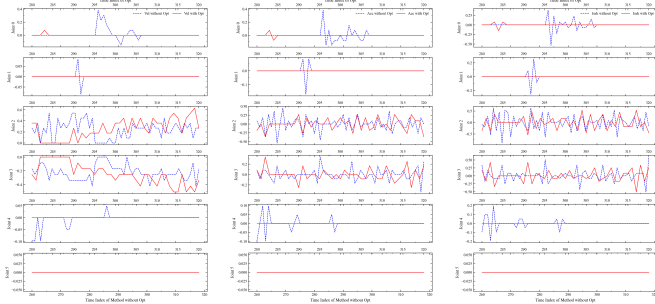
““latex ““

As shown in Table II, latent optimization reduces the average mean absolute jerk about 14.61% and the average mean absolute acceleration about 10.84%. It is consistent with Eq. 18, where higher-order smoothness, especially jerk, is emphasized. The strongest improvement appears in DoF 4, while DoF 0 and DoF 1 also improve across jerk, acceleration, and velocity. Fig. 6 further shows that the optimized trajectories suppress spikes and oscillations during insertion, producing smoother motion traces. Figure 5 illustrates more obviously how the choice of latent variable z affects execution in the maneuver task. When z is sampled randomly, actions can remain task-successful.

However, the mean absolute velocity increases from 2.29 to 2.84 with 24.02%, which is primarily driven by DoF 2 and DoF 3. Despite the higher average speed, both DoF 2



Fig. 5: Inserting under different latent choices. Each row shows a time-lapse of a single rollout. Top: z is randomly sampled, which may induce a horizontal bias in the executed motion. As a result, the task may still be successful, but the resulting circle formed by the blocks can exhibit a noticeable lateral offset. Bottom: z is optimized, producing a more stable motion and a final block configuration with reduced horizontal displacement.



(a) Per-DoF velocity: w/o optimization (blue) vs. optimized (red). (b) Per-DoF acceleration: w/o optimization (blue) vs. optimized (red). (c) Per-DoF jerk: w/o optimization (blue) vs. optimized (red).

Fig. 6: Per-DoF comparison of velocity, acceleration, and jerk with (red) and without (blue) optimization over the insertion. Only inter-frame differences to demonstrate the trend, without the physical time derivative.

and DoF 3 still achieve reductions in the jerk. It could suggest that the optimizer may accept higher velocity in some joints to more effectively suppress acceleration or jerk while maintaining feasibility and achieving the required insertion motion. Additionally, it is expected because z parameterizes an implicit mapping from latent space to the full multi-DoF horizon. Consequently, optimizing z cannot guarantee monotonic improvement for every joint independently and uniformly. Instead, the optimizer may reduce the composite objective in Eq. 18 by redistributing smoothness cost across joints, such as allowing slightly higher velocities in some DoFs while substantially suppressing jerk spikes in others, so long as the overall trajectory remains feasible and achieves the required insertion pose.

3) Feasibility of Latent-Space Optimization: We test whether latent-space optimization produces successful objective-driven updates by varying the jerk weight w_{jerk} while keeping all other loss weights fixed as 0.1. For each optimization iteration in each action sequence plan, we compute

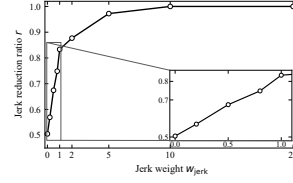


Fig. 7: Jerk Descent Success Ratio under Different Weights

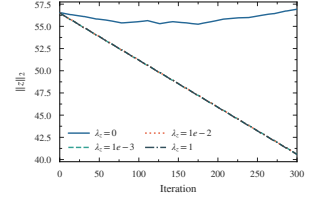


Fig. 8: z Norm Across Optimization Iterations for Different Prior Weights

a scalar jerk cost J_{jerk} from the decoded joint trajectory and record whether this cost decreases relative to the previous iteration. We treat an update as successful and objective-driven if it produces a descent step on the jerk term, i.e., J_{jerk} is lower than in the previous iteration. We then report the descent success ratio of Jerk term J_{jerk} , defined as the fraction of all valid iteration-to-iteration updates across all plans for which the J_{jerk} decreases. The metric measures the effectiveness of latent optimization in consistently reducing the targeted jerk term during iterative refinement.

As shown in Fig. 7, increasing w_{jerk} monotonically increases jerk reduction ratio in the updating process, with a clear saturation effect. When $w_{\text{jerk}} = 0$, the successful descent rate is ≈ 0.5 , indicating that the J_{jerk} is almost randomly updating. There is no systematic improvement and driven by jerk cost term as expected. However, increasing w_{jerk} makes nearly every update a descent step on J_{jerk} . It demonstrates that the latent variable update is not arbitrary, but instead follows the user-specified objective. Additionally, the saturation further suggests that once the jerk term dominates the optimization direction, increasing its weight beyond a certain level yields diminishing returns in terms of descent consistency. In summary, it verifies that the proposed latent-space optimization framework can produce objective-driven updates, and that the optimization behavior remains controllable through the choice of loss weighting.

4) Ablation Study of Penalty Terms: To evaluate the prior penalty, we initialize z to 10 which is far from the latent mean and vary the weight λ_z in Eq.16. With λ_z is 0, latent variable z cannot decrease and the planner remains in low-probability regions; in real execution, it causes failure during *Approach-and-Grasp*, where the arm jitters above block A and never secures a grasp. For any non-zero penalty, z is pulled back toward the mean so that after 1-2 attempts of grasp action chunk, the full task succeeds despite minor trajectory jitter. This ablation confirms that the prior penalty is crucial for feasibility-preserving latent optimization by steering the search away from low-probability latent regions that are unlikely to satisfy the implicit constraints, especially under poor initialization.

We further observe switching from $\lambda_z = 0$ to a small non-zero value, such as 10^{-3} produces a qualitative change in the optimization dynamics. In contrast, increasing the weight further from 10^{-3} to 1 yields only marginal differences in both the $\|z\|_2$ updating and the qualitative task outcome. It

suggests that the dominant effect can come from *activating* the prior-aware objective term, and that once the penalty term is present, the overall dynamics may become relatively insensitive to its exact scale. A possible explanation is that the relative scaling between the task objective and the prior term can be partially compensated by Adam-style adaptive step normalization. In conclusion, a small non-zero λ_z is sufficient to obtain the feasibility-preserving effect in practice.

5) *Efficiency Analysis*: We optimize a 32-dimensional latent variable $z \in \mathbb{R}^{32}$, which is substantially smaller than directly optimizing a full action chunk in the original action space. In our setup, each action is 6-DoF and the chunk length is 60, yielding $6 \times 60 = 360$ optimization variables in the original space. The reparameterized decoder contains approximately 37.695M parameters, and one optimization iteration, including forward decoding plus backpropagation through the frozen decoder to update z , takes about 0.0196s on an RTX 3090. After the first iteration, it takes about 0.0160s, which is less than the first iteration. It suggests, to some extent, that our warm-start strategy can reduce the computational cost of optimization.

Despite the reduced optimization dimension, latency can still arise due to iterative refinement. In practice, we optimize smoothness objectives at least 15 iterations, resulting in an online planning time of roughly $\gtrsim 0.24$ s. Its delay introduces a non-negligible pause in the online optimization, which is undesirable for reactive manipulation. It could be solved by buffer assignment or by a temporal ensemble policy with action chunk [10], [22].

V. CONCLUSIONS

Robotic manipulation can be posed as constrained optimization, but in contact-rich tasks, the key feasibility constraints are implicit, hybrid, and task-dependent, making them hard to model and maintain manually. To address this, we learn a data-driven reparameterization of the feasible action space from demonstrations, enabling efficiency unconstrained optimization planning. Additionally, our method does not require explicit geometric models or object CAD representations. Instead, it relies solely on RGB visual feedback and current robot state, allowing the robot to interact with objects in real time while supporting user-defined objective preferences during optimization. We evaluate the approach on a constraint-rich manipulation task and show that feasibility is largely preserved. Moreover, latent-space refinement improves motion quality, reducing insertion-phase jerk by 14.61% and acceleration by 10.84%. However, iterative latent refinement can still introduce noticeable action delays when many gradient steps are required, despite the reduced decision dimension. Besides, the approach depends on the coverage of the demonstration dataset. In the future, we plan to improve inference architecture such as running asynchronous inference to reduce the delay in online optimization. We also aim to scale to combinatorially growing, task-dependent constraints by integrating active data collection and online adaptation to continually expand feasible-action coverage across objects and tasks.

ACKNOWLEDGMENT

This research was supported by JSPS KAKENHI Scientific Research (S) Grant Number 22H05002. And thank We also acknowledge Yanbo Wang and Yongqiang Dou for their insightful discussions and generous support in providing the hardware used in this research.

REFERENCES

- [1] J.-P. Sleiman *et al.*, “Contact-implicit trajectory optimization for dynamic object manipulation. in 2019 IEEE,” in *RSJ International Conference on Intelligent Robots and Systems (IROS)*, 6814–6821.
- [2] M. Zhang *et al.*, “Simultaneous trajectory optimization and contact selection for multi-modal manipulation planning,” *arXiv preprint arXiv:2306.06465*, 2023.
- [3] J. Lee *et al.*, “Uncertain pose estimation during contact tasks using differentiable contact features,” *arXiv preprint arXiv:2305.16778*, 2023.
- [4] A. Brohan *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [5] B. Zitkovich *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan *et al.*, Eds., vol. 229. PMLR, 06–09 Nov 2023, 2165–2183.
- [6] D. Driess *et al.*, “Deep visual heuristics: Learning feasibility of mixed-integer programs for manipulation planning,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, 9563–9569.
- [7] G. Souto *et al.*, “Learning equality constraints for motion planning on manifolds,” in *Conference on Robot Learning*. PMLR, 2021, 2292–2305.
- [8] N. Hansen *et al.*, “Td-mpc2: Scalable, robust world models for continuous control,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [9] Z. Zhao *et al.*, “A survey of optimization-based task and motion planning: From classical to learning approaches,” *IEEE/ASME Transactions on Mechatronics*, vol. 30, no. 4, 2799–2825, 2025.
- [10] T. Z. Zhao *et al.*, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
- [11] N. M. M. Shafullah *et al.*, “Behavior transformers: Cloning k modes with one stone,” in *Thirty-Sixth Conference on Neural Information Processing Systems*, 2022.
- [12] P. Florence *et al.*, “Implicit behavioral cloning,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust *et al.*, Eds., vol. 164. PMLR, 08–11 Nov 2022, 158–168.
- [13] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *arXiv preprint arXiv:2303.04137*, 2023.
- [14] A. Y. Ng and S. Russell, “Algorithms for inverse reinforcement learning,” in *Proceedings of the Seventeenth International Conference on Machine Learning*. Morgan Kaufmann, 2000, 663–670.
- [15] P. Abbeel and A. Y. Ng, “Apprenticeship learning via inverse reinforcement learning,” in *Machine Learning, Proceedings of the Twenty-first International Conference (ICML 2004)*. ACM, 2004.
- [16] C. Finn *et al.*, “Guided cost learning: Deep inverse optimal control via policy optimization,” in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, 49–58.
- [17] G. Zhou *et al.*, “Diffusion model predictive control,” *Transactions on Machine Learning Research*, May 2025.
- [18] C. Pan *et al.*, “Model-based diffusion for trajectory optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [19] W. Zhao *et al.*, “Vlmpc: Vision-language model predictive control for robotic manipulation,” *arXiv preprint arXiv:2407.09829*, 2024.
- [20] Y. Hamasuna *et al.*, “On mahalanobis distance based fuzzy c-means clustering for uncertain data using penalty vector regularization,” in *2011 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE 2011)*. IEEE, 2011, 810–815.
- [21] R. Cadene *et al.*, “Lerobot: An open-source library for end-to-end robot learning,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [22] K. Black *et al.*, “Real-time execution of action chunking flow policies,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.